UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación

TRABAJO PRÁCTICO INTEGRADOR

Food Store - Segunda entrega

Capa de datos: Unidades 4 a 8

PostgreSQL 18.6 · MongoDB 8.0.34 · SQLite 3.54

Materia: Base de Datos II

Carrera: Tecnicatura Universitaria en Programación (UTN)

Integrantes:

Lemus Nahuel

Anglat Juan

Elias Juan Cruz

Repositorio: github.com/hero21lb/foodstore-gestor

Año 2026

Materia: Base de Datos II - UTN FRM Proyecto: Food Store Integrantes: Lemus Nahuel, Anglat Juan, Elias Juan Cruz Repositorio: https://github.com/hero21lb/foodstore-gestor

1. Alcance y arquitectura

La segunda entrega cierra la capa de datos del proyecto con tres motores:

Motor	Rol	Autoridad
PostgreSQL 18.6	Catálogo, precios, stock, pedidos, ventas, roles y auditoría.	Fuente de verdad de precio, stock y venta.
MongoDB 8.0.34	Colección `foodstore_u7.pedidos`: pedido documental con líneas embebidas y cliente seudonimizado.	Fuente del documento de pedido para consulta NoSQL; no reemplaza la venta relacional.
SQLite 3.54	`tablet_u8.db`: catálogo de lectura, borradores y cola de sincronización.	No manda: propone y simula; PostgreSQL decide.

La base relacional de trabajo es `foodstore` (copia de `railway`). Para U8 se usó una base nueva `foodstore_u8_lab`; se eliminó al terminar y no se tocó `railway`.

2. Unidad 4 - Normalización avanzada y redundancia

2.1 FNBC

Relación original `R(LoteID, DepositoID, ResponsableControlID)` con `ResponsableControlID -> DepositoID`. Se descompone en:

- `R1(ResponsableControlID, DepositoID)`.
- `R2(LoteID, ResponsableControlID)`.
- Vista de compatibilidad `vw_control_lote_almacen`.

La migración se ejecuta en una transacción y se aborta si un responsable aparece en dos depósitos. La dependencia `(LoteID, DepositoID) -> ResponsableControlID` se protege con un trigger y advisory lock por lote.

2.2 Desnormalización controlada

Se agregaron a `detalle_pedido` las columnas `fecha_venta_u4`, `categoria_id_u4`, `categoria_nombre_u4` y `subtotal_u4`, con backfill transaccional y triggers de sincronización para cambios de línea, fecha de pedido, categoría de producto y nombre de categoría. El índice `idx_detalle_top_categorias_u4` cubre fecha y categoría.

2.3 Resultados

Prueba	Resultado
`EXCEPT ALL` bidireccional original vs. vista	0 filas
Vista reconstruida	`(501,30,801)`, `(502,30,801)`, `(503,31,802)`
Segundo responsable para el mismo lote/depósito	Rechazado por trigger
Top 5 antes / después	14,440 ms / 1,280 ms (captura); mediana 14,576 / 1,230 ms
Auditoría de columnas precalculadas	0 diferencias

Evidencia: `food-store-tp-u4/docs/evidencia_u4.md`, `food-store-tp-u4/TP_U4_FNBC_Desnormalizacion_FoodStore_Entregable.pdf`.

3. Unidad 5 - Administración y seguridad

3.1 Roles y mínimo privilegio

Se definieron roles `NOLOGIN` (`rol_app_lectura`, `rol_app_escritura`, `rol_soporte`, `rol_reportes`), dos roles `LOGIN` (`app_web`, `admin_datos`) y un propietario `NOLOGIN` para funciones de autenticación (`foodstore_auth_owner`). Ningún rol funcional lee `password_hash`; `app_web` no concede roles ni borra ventas. Se revocaron permisos de `PUBLIC`, se cerraron tablas futuras y se revocó `EXECUTE` global sobre funciones.

3.2 Autenticación y auditoría

`fn_autenticar` y `fn_resetear_password` son `SECURITY DEFINER`, con `search_path` fijo y propietario sin login. Las contraseñas usan bcrypt de `pgcrypto`; los hashes nunca se devuelven. La auditoría global se activó y se probó con un simulacro de acceso indebido:

- 3 fallos de autenticación, 1 éxito.
- Lectura real de las 18 filas sintéticas de `usuario`, sin hashes.
- `GRANT admin_datos TO app_web`: rechazado con SQLSTATE 42501.
- `pg_has_role` confirmó que la membresía no existe.
- Se verificó que `RAISE LOG` no exponga parámetros sensibles.

3.3 Decisión humana

Se aceptó el esquema de mínimo privilegio y se descartaron las propuestas de `CREATEDB`/`CREATEROLE` y de lectura automática de tablas futuras. OpenCode se intentó usar para auditar el SQL, pero devolvió HTTP 400 por un esquema MCP incompatible; el error quedó en `docs/opencode_auditoria.jsonl` y no se inventó una auditoría. La decisión final es del equipo.

Evidencia: `food-store-tp-u5/`, `TP_U5_Administracion_Seguridad_FoodStore_Entregable.pdf`.

4. Unidad 6 - Mantenimiento

Plan: respaldo custom diario de PostgreSQL, copia diaria de `tablet_u8.db` y archivo de las instantáneas CSV. La restauración se prueba en una base nueva, comparando conteos y `md5` de las cinco tablas y verificando que no haya precios o stock inválidos.

Simulacro real: dump `foodstore_20261020T205205_768821Z.dump` restaurado en `foodstore_restore_test`.

Métrica	Resultado
RTO	32,224 s

Métrica	Resultado
Antigüedad del respaldo al iniciar	263,558 s
Conteos restaurados	20 / 1000 / 5000 / 20000 / 20000
md5 por tabla	Coincidentes
Precios/stock inválidos	0
Restricciones restauradas	18

Se descartó `VACUUM FULL` periódico; se usa `ANALYZE` cuando las estadísticas lo justifican. El informe post-incidente está en `food-store-tp-u6/logs/restore_resultado.json` y en el PDF de U6.

Evidencia: `food-store-tp-u6/`, `TP_U6_Mantenimiento_FoodStore_Entregable.pdf`.

5. Unidad 7 - NoSQL

Modelo documental: un documento por pedido en `foodstore_u7.pedidos`. `_id` conserva el BIGINT como BSON Long; `cliente` guarda ID y alias sintético; `lineas` embebe producto, nombre de catálogo, cantidad, precio histórico y subtotal; `total` usa `Decimal128`. No se exportan nombres, apellidos, mails ni teléfonos.

Migración: 5 pedidos, con estados sintéticos asignados en PostgreSQL antes de migrar (PENDIENTE, CONFIRMADO, ENTREGADO, CANCELADO, CONFIRMADO). Se verificó igualdad de documentos y hashes antes/después.

Consulta	PostgreSQL	MongoDB
Todos	1,2,3,4,5	1,2,3,4,5
Estado CONFIRMADO	2,5	2,5
Producto 2	1	1
Total > 300	2,3,4	2,3,4

La comparación usa los mismos cinco pedidos en ambos motores. MongoDB se elige para la ficha/consulta documental; PostgreSQL sigue siendo la fuente de la venta y del precio autoritativo.

Evidencia: `food-store-tp-u7/consultas_mongodb.js`, `food-store-tp-u7/evidencias/verificacion.json`, `TP_U7_NoSQL_FoodStore_Entregable.pdf`.

6. Unidad 8 - Base móvil e integración

6.1 Esquema SQLite

Cuatro tablas `STRICT` con `PRAGMA foreign_keys = ON`:

• `producto_cache`: id, nombre, precio en centavos, stock visto, disponible,

eliminado, fecha de copia.

• `pedido_borrador`: UUID local, usuario, forma de pago, fecha.

• `item_borrador`: PK compuesta, FK a borrador y caché, cantidad > 0.

• `cola_sync`: UUID de solicitud, estado y motivo.

6.2 Sincronización y semáforo

`v_snapshot_movil` exporta solo catálogo a CSV. La tablet lo importa en `snapshot_servidor` y carga `producto_cache`. El semáforo evalúa solo `PENDIENTE`:

• RECHAZADO: producto inexistente, dado de baja, no disponible o stock

insuficiente.

• CONFLICTO: cambió el precio.

• SIMULADO_APROBADO: todo coincide; no es una venta.

6.3 Casos

Caso	Resultado
C1 catálogo offline	5 productos desde `producto_cache`, sin PostgreSQL
C2 precio	`CONFLICTO` "visto 105000, ahora 110000"; al aceptar, nueva solicitud `SIMULADO_APROBADO`
C3 stock	`RECHAZADO` "pedido 5, disponible 2"
C4 baja lógica	`RECHAZADO` "dado de baja"; fila queda con `eliminado=1`, catálogo no la muestra
C5 sin cambios	`SIMULADO_APROBADO`
C6 mismo UUID	error de PK; no duplica ni reevalúa
C7 cantidad 0 / producto inexistente	CHECK/FK; 0 filas nuevas

Control final en PostgreSQL: 5 pedidos, 7 detalles, stock 220, mismos `md5`. La tablet no creó pedidos ni movió stock.

Code review final: 6 hallazgos (H-01 a H-06), con H-02 corregido y H-05 descartado. Ver `food-store-tp-u8/docs/code_review_u8.md`.

Evidencia: `food-store-tp-u8/u8/resultados/evidencia_u8.txt`, `food-store-tp-u8/TP_U8_Tablet_FoodStore_Entregable.pdf`.

7. Integración y decisiones

Decisión	Alternativa descartada	Motivo
PostgreSQL manda precio/stock/venta	Confirmar venta desde SQLite	La tablet no puede reservar ni validar stock real.
SQLite guarda solo una foto	Réplica bidireccional	La consigna pide simulación y la tablet no debe escribir el servidor.
Precio en centavos	REAL/decimal en SQLite	Comparaciones exactas y sin redondeo.
MongoDB para el documento de pedido	Poner todo en PostgreSQL	Prueba práctica NoSQL y consultas documentales.

Decisión	Alternativa descartada	Motivo
Respaldo custom + restore probado	Copia de archivos sin prueba	El respaldo solo vale si se restaura.

8. IA, Kiro y Git

• Specs: `food-store-tp-u4/specs/`, `food-store-tp-u5/docs/kiro_roles_spec/`,

`food-store-tp-u6/SPEC_U6.md`, `food-store-tp-u7/AGENTS.md` + README, `food-store-tp-u8/docs/requirements.md` + `design.md` + `tasks.md`.

• Bitácoras/DUIA: `food-store-tp-u5/BITACORA.md`,

`food-store-tp-u7/asistencia_ia.md`, `food-store-tp-u8/docs/duia_u8.md`, y `TPI/DUIA.md`.

• OpenCode se usó para generar y revisar; en U5 el intento de auditoría falló

por HTTP 400 y quedó registrado sin inventar resultados.

• Todo el material está versionado en Git. El repositorio incluye los scripts,

PDFs, logs y evidencias de las unidades 4 a 8.

9. Cómo reproducir

1. PostgreSQL: `food-store-tp-u1-u3/sql/schema_completo.sql`, `data.sql`, `restricciones_tp2.sql`, `indices.sql`, `indices_u3.sql`, `views.sql`, `objetos_programables_tpi.sql`. 2. U4: `food-store-tp-u4/sql/tp_fnbc_control_lote.sql` y `tp_desnormalizacion_top_categorias.sql`. 3. U5: `food-store-tp-u5/sql/roles.sql` y `laboratorio_u5.py`. 4. U6: `food-store-tp-u6/backup_foodstore.ps1` y `restore_test.ps1`. 5. U7: `food-store-tp-u7/laboratorio_u7.py` y `consultas_mongodb.js`. 6. U8: `food-store-tp-u8/README.md` y `food-store-tp-u8/u8/run_u8.mjs`.

10. Gestor para la defensa

El gestor web del proyecto incorpora una vista Laboratorio TPI con una tarjeta por unidad (U1 a U8). Cada tarjeta ejecuta una prueba real contra PostgreSQL, MongoDB o SQLite y muestra el resultado en pantalla. También incluye una vista Defensa con el guion de la charla y las preguntas probables.

• URL: https://foodstore-gestor-production.up.railway.app/

• Repositorio: https://github.com/hero21lb/foodstore-gestor

• Guion completo: `TPI/GUIA_DEFENSA.md`